

Server PostgreSQL Integration Suite — Executed Against Real Rootless Postgres

Revision: 1 Last modified: 2026-07-09T19:45:00Z

Scope

Stream M2 task: previously the server's `-tags integration` Postgres suite (`server/internal/store/*_integration_test.go`, `server/internal/rollout/*_integration_test.go`) was known to type-check (`go vet -tags integration/go build -tags integration clean`) but had **never been executed against a live PostgreSQL** — a Stream V audit finding. `architecture.md` §4 names the `pgx/PostgreSQL` Repository the production persistence target, so this suite is the only test evidence that the production path (as opposed to the in-memory MVP store that ships wired by default) actually behaves correctly against a real database.

This run closes that gap: the suite was executed twice, end-to-end, against a real, rootless-podman-booted PostgreSQL 16, with the container brought up and torn down entirely by the test code itself (no manual `podman run` needed — see “Boot method” below).

Result summary

RAN — did not SKIP. All Postgres integration tests **PASSED** on both runs. No findings, no product bugs discovered.

Run	Command	Result
1	<code>go test -tags integration ./... -count=1 -v (full server module)</code>	PASS — 15/15 packages ok, 0 FAIL
2	<code>go test -tags integration -race</code>	

Run	Command	Result
	-count=1 ./internal/ store/... ./ internal/rollout/... -v	PASS — 2/2 packages ok, 0 FAIL, 0 DATA RACE reports

Postgres-backed integration tests executed (both internal/store and internal/rollout packages boot their OWN dedicated Postgres container on distinct host ports — 55432 and 55445 respectively — so they can run in parallel without contention, serialized only by the pg_itest_lock_test.go flock per §11.4.119):

Test	Package	Run 1 (no -race)	Run 2 (- race)
TestPostgresRepositoryContract_Integration	internal/ store	PASS 4.28s	PASS 8.20s
TestPostgresCoveragePaths_Integration	internal/ store	PASS 8.05s	PASS 8.44s
TestPostgresFabricRegistry_Integration	internal/ store	PASS 8.25s	PASS 8.15s
TestPostgresDriverFaults_Integration (+3 subtests: query_connection_failure, mid_iteration_rows_err, pool_acquire_failure)	internal/ store	PASS 4.87s	PASS 9.06s
TestPostgresStoreScenario_Integration	internal/ rollout	PASS 8.10s	PASS 8.82s
TestPostgresStoreCoveragePaths_Integration	internal/ rollout	PASS 4.06s	PASS 8.44s
TestPostgresStoreDriverFaults_Integration (+3 subtests: connection_failure, mid_transaction_commit_failure, pool_acquire_failure)	internal/ rollout	PASS 4.12s	PASS 8.39s

Package-level timings:

Run 1 (full suite, no -race):

```
ok      .../server/internal/store 25.469s
ok      .../server/internal/rollout 16.294s
(13 other non-DB packages also ok, 0 FAIL — full module green)
```

Run 2 (-race, store+rollout only):

```
ok      .../server/internal/store    30.987s
ok      .../server/internal/rollout    26.670s
10.02user 5.01system 0:33.96elapsed 44%CPU
```

grep -c "DATA RACE" on the -race log: 0.

Boot method — rootless podman, no manual step

The suite is self-booting: postgres_integration_test.go / postgres_coverage_integration_test.go / postgres_fabric_integration_test.go / postgres_fault_integration_test.go in both packages call runtime.AutoDetect(ctx) then compose.NewDefaultOrchestrator + boot.NewBootManager from the project's own submodules/containers (digital.vasic.containers, wired via server/go.mod's replace digital.vasic.containers => ../containers) — the constitution's mandated on-demand-infra path (§11.4.76/§11.4.161), never an ad-hoc podman run. No manual container boot was performed by this stream; the Go test process invoked the orchestrator itself for every run.

Compose files consumed: server/deploy/postgres.compose.yml (host port 55432, project internal/store) and server/deploy/postgres-rollout.compose.yml (host port 55445, project internal/rollout), both docker.io/library/postgres:16-alpine.

Rootless proof (captured before the run):

```
$ podman info | grep -iE "rootless|graphRoot|runRoot"
  rootlessNetworkCmd: pasta
  security:
    rootless: true
  graphRoot: /home/milos/.local/share/containers/storage
  runRoot: /run/user/1000/containers

$ id
uid=1000(milos) gid=1000(milos) groups=1000(milos),... (no root,
no sudo)

$ podman --version
podman version 5.7.1
$ podman-compose --version
podman-compose version 1.5.0
```

No sudo was used anywhere in this stream. No rootful docker was used. The postgres:16-alpine image was already present in local podman image storage (podman images showed it cached), so no network pull was required for this run — a genuinely offline-capable rootless boot.

Real captured boot-summary lines from the test's own logging (proving the container actually started, not a stub):

```
postgres_integration_test.go:48: container runtime: podman
postgres_integration_test.go:81: boot summary: started=1
discovered=0 failed=0 in 648.75759ms
postgres_coverage_integration_test.go:49: container runtime:
podman
postgres_coverage_integration_test.go:80: boot summary: started=1
discovered=0 failed=0 in 647.637616ms
postgres_fabric_integration_test.go:37: container runtime: podman
```

DSN / migration — no manual step required

Each test hardcodes its own dedicated DSN (distinct ports keep the two packages non-contending):

- internal/store: postgres://helix:helix@localhost:55432/helix_ota?sslmode=disable
- internal/rollout: postgres://helix:helix@localhost:55445/helix_ota?sslmode=disable

No separate DATABASE_URL env var needed to be set by this stream — the integration tests are self-contained and construct/consume their own DSN constants. Schema migration is driven by the tests themselves against the freshly-booted container (verified live by TestPostgresFabricRegistry_Integration, which asserts the fabric_nodes / fabric_targets / fabric_leases / fabric_runs / fabric_evidence tables and the uq_fabric_lease_active unique partial index actually exist post-migration in the real database — not asserted against a mock).

Real findings surfaced by the suite (proof it exercises the real driver, not a stub)

TestPostgresDriverFaults_Integration /

TestPostgresStoreDriverFaults_Integration genuinely kill the live TCP connection to the real Postgres mid-query/mid-transaction and assert the pgx driver surfaces a clean, categorized error rather than a panic or silent hang. Captured real driver output (not synthesized):

```
postgres_fault_integration_test.go:102: ListDevices surfaced clean driver error: unexpected EOF
```

```
postgres_fault_integration_test.go:106: ListReleases surfaced clean driver error: failed to connect to `user=helix database=helix_ota`: 127.0.0.1:46025 (127.0.0.1): failed to receive message: unexpected EOF
```

```
postgres_fault_integration_test.go:195: mid-iteration rows.Err captured cleanly after 66 rows: unexpected EOF
```

```
postgres_fabric_integration_test.go:122: EVIDENCE-CHECK: raw 0-byte INSERT REJECTED by DB CHECK -> ERROR: new row for relation "fabric_evidence" violates check constraint
```

```
"fabric_evidence_byte_size_check" (SQLSTATE 23514)
```

```
postgres_fabric_integration_test.go:101: LEASE-UNIQUENESS: second lease L2 on t1 REJECTED by DB -> store: conflict (mapped to ErrConflict)
```

These are genuine DB-enforced constraints (a real CHECK constraint rejection with a real Postgres SQLSTATE 23514, a real partial-unique-index conflict) — not achievable against an in-memory fake. This is exactly the production-path assurance the suite exists to provide.

Findings / product bugs

None. No test failed, no data race was detected, and no unexpected behavior was observed against the real database on either run. The production pgx/PostgreSQL persistence path (architecture.md §4) is confirmed working end-to-end by this execution, including its documented fault-handling behavior under injected connection loss and its schema constraints (fabric lease uniqueness, evidence byte-size check).

Clean teardown

`t.Cleanup(func() { _ = mgr.Shutdown(context.Background()) })` is registered by each test and runs at the end of every test (both the success path here and any future failure path), tearing the compose project down. Verified post-run:

```
$ podman ps -a --format "{{.Names}}\t{{.Status}}" | grep -iE
"helix.ota|55432|55445|postgres.compose"
(no leftover integration containers)
```

No orphaned containers, networks, or volumes from either run remained on the host after the suite completed (checked after both Run 1 and Run 2).

Environment

- go version go1.26.4-X:nodwarf5 linux/amd64
- podman version 5.7.1, podman-compose version 1.5.0, rootless (security.rootless: true)
- Working directory: /home/milos/Factory/projects/tools_and_research/helix_ota/server
- Full raw logs (kept in scratchpad, not committed — regeneratable by re-running the commands above):
 - Run 1: `go test -tags integration ./... -count=1 -v`
 - Run 2: `go test -tags integration -race -count=1 ./internal/store/... ./internal/rollout/... -v`

Conclusion

The production Postgres persistence path is now proven with real, captured, DB-backed evidence rather than type-checked-only assurance. No conductor action required — no bugs found, no product code touched by this stream.